

FACULDADE COTEMIG
CURSO DE SISTEMAS DE INFORMAÇÃO
DISCIPLINA: FUNDAMENTOS TEÓRICOS DA COMPUTAÇÃO

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DE MÁQUINAS ABSTRATAS
DESENVOLVIMENTO DE COGNITORES DE LINGUAGENS: AFD, AP E MT

Grupo de Alunos:

Rafael Soares Almeida Fonseca
72400749

Luiz Felipe de Souza Cassimiro
72301139

Professor: Júlio César da Silva

Data de Entrega: 10/06/2026

Belo Horizonte - MG

1. INTRODUÇÃO

O estudo da Teoria da Computação e das Linguagens Formais fundamenta-se na classificação de problemas de acordo com sua complexidade intrínseca, mapeada historicamente pela Hierarquia de Chomsky. Para validar a equivalência prática entre os modelos teóricos matemáticos e a computação programática concreta, este projeto apresenta o desenvolvimento e a análise de um sistema simulador em linguagem C#.

O software unificado desenvolvido é capaz de processar e avaliar cadeias de caracteres por meio de três classes de máquinas abstratas distintas: um Autômato Finito Determinístico (AFD), um Autômato de Pilha (AP) em suas vertentes determinística e não-determinística, e uma Máquina de Turing (MT). Cada modelo foi projetado para responder a restrições gramaticais específicas, demonstrando de forma empírica o incremento de poder computacional à medida que estruturas de memória avançadas são incorporadas ao controle de estados.

2. AUTÔMATO FINITO DETERMINÍSTICO (AFD)

2.1 Descrição da Linguagem (L1)

O AFD implementado foi configurado para reconhecer a linguagem regular L1 sobre o alfabeto $\Sigma = \{a, b\}$, definida como:

$$L1 = \{ w \in \{a, b\}^* \mid w \text{ termina com o sufixo } ab \}$$

A linguagem exige que, independentemente do prefixo processado, a sequência de computação termine obrigatoriamente com a leitura consecutiva dos símbolos 'a' e 'b'. Cadeias vazias ou que terminam em 'a' ou 'bb' são formalmente rejeitadas.

2.2 Definição Formal

O autômato é definido formalmente pela quintupla $M1 = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q (Conjunto de Estados): $\{q_0, q_1, q_2\}$
- Σ (Alfabeto de Entrada): $\{a, b\}$
- q_0 (Estado Inicial): q_0

- F (Conjunto de Estados de Aceitação): $\{q_2\}$
- δ (Função de Transição): Definida mapeando $Q \times \Sigma \rightarrow Q$ conforme a tabela abaixo:

Estado atual	Lê 'a'	Lê 'b'	Semântica do Estado
$\rightarrow q_0$	q_1	q_0	Estado inicial / Nenhuma subcadeia relevante lida.
q_1	q_1	q_2	Último caractere lido foi um 'a' (potencial início de sufixo).
$*q_2$	q_1	q_0	Sufixo 'ab' reconhecido com sucesso (aceitação).

2.3 Decisões de Implementação

A função de transição δ foi codificada utilizando a estrutura de dados orientada a objetos `Dictionary<(string estado, char simbolo), string>`. Essa abordagem confere à simulação uma complexidade de tempo constante $O(1)$ para cada busca de transição.

O programa implementa o conceito de Função Total, validando previamente se cada caractere da cadeia pertence a Σ . Caso um símbolo inválido seja detectado, a computação interrompe-se imediatamente, classificando a cadeia como rejeitada por erro de alfabeto. O rastro de execução é dinamicamente armazenado em uma lista genérica (`rastroAtual`), garantindo a rastreabilidade completa dos estados visitados. Adicionalmente, foi criada uma rotina de parsing customizada por meio de Expressões Regulares (Regex) para ler e carregar dinamicamente novas especificações de AFDs contidas em arquivos estruturados em formato JSON (`afd.json`).

2.4 Exemplos de Execução

A tabela abaixo documenta o comportamento do simulador ao processar arquivos do lote de testes padrão (entradas.txt):

Cadeia (w)	Rastro de Estados Percorrido	Resultado	Justificativa Teórica
ab	$q_0 \rightarrow q_1 \rightarrow q_2$	ACEITA	Termina exatamente com o sufixo 'ab'.
aab	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$	ACEITA	O segundo 'a' mantém em q_1 , finalizando em 'b' para q_2 .
ababab	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2$	ACEITA	Múltiplas oscilações que encerram com sucesso em q_2 .
ba	$q_0 \rightarrow q_0 \rightarrow q_1$	REJEITA	Termina no estado q_1 , que não pertence a F.
b	$q_0 \rightarrow q_0$	REJEITA	Termina no estado inicial q_0 .
(Vazia)	q_0	REJEITA	A cadeia vazia não atinge o estado de aceitação.

2.5 Questões Reflexivas do Modelo

1. Por que o AFD não consegue reconhecer linguagens com contagem simétrica como $a^n b^n$?

O AFD possui um controle de memória estritamente finito limitado ao seu número cardinal de estados $|Q|$. Para reconhecer $a^n b^n$, a máquina precisaria "lembrar" a quantidade exata de símbolos 'a' lidos para posteriormente exigir a mesma quantidade de 'b's. Como n é arbitrário e infinito, um AFD demandaria infinitos estados para armazenar essa contagem, o que viola sua definição matemática fundamental.

2. Como a leitura total da cadeia e a completude da função de transição influenciam a confiabilidade do sistema?

A completude garante que o autômato nunca entre em um estado de indefinição ou travamento operacional. Ao assegurar que $\forall q \in Q, \forall a \in \Sigma$ exista um destino explícito, o sistema elimina comportamentos erráticos. A leitura total impede que subcadeias válidas intermediárias gerem falsos positivos, certificando que a validação ocorra exclusivamente sob a configuração final pós-consumo de dados.

3. AUTÔMATO DE PILHA (AP)

3.1 Descrição das Linguagens (L2 e L3)

A aplicação expande o escopo teórico simulando duas linguagens livres de contexto clássicas:

- L2 (Determinística - APD): $L2 = \{ a^n b^n \mid n \geq 1 \}$. Exige uma sequência estrita de 'a's seguida por uma quantidade idêntica de 'b's.
- L3 (Não-Determinística - APN): Palíndromos gerais sobre $\{a, b\}^*$, onde a cadeia lida da esquerda para a direita é idêntica à lida de modo inverso ($w = w^R$).

3.2 Definição Formal

Os autômatos de pilha são modelados pela sétupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$. Ambos os casos foram implementados com Reconhecimento por Pilha Vazia, o que desobriga a especificação de estados finais ativos ($F = \emptyset$).

Configuração para L2 (Caso Determinístico):

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{A, Z\}$ (onde Z é o delimitador inicial da base da pilha)

- $\delta(Q \times (\Sigma \cup \{\lambda\}) \times \Gamma) \rightarrow Q \times \Gamma^*$:

$\delta(q_0, a, Z) = (q_0, AZ)$ — Empilha A sobre Z
 $\delta(q_0, a, A) = (q_0, AA)$ — Empilha A sobre A
 $\delta(q_0, b, A) = (q_1, \lambda)$ — Transiciona consumindo um A via desempilhamento
 $\delta(q_1, b, A) = (q_1, \lambda)$ — Continua desempilhando A para cada b
 $\delta(q_1, \lambda, Z) = (q_2, \lambda)$ — Esvazia o símbolo base Z por movimento vazio, aceitando

Configuração para L3 (Caso Não-Determinístico):

O modelo gerencia o não-determinismo na identificação do ponto médio (eixo de simetria) do palíndromo. No estado q_0 , para cada símbolo lido, o autômato bifurca-se em três ramificações lógicas simultâneas:

- Empilhar o caractere atual e permanecer em q_0 (processamento da primeira metade).
- Avançar para o estado de casamento de padrões q_1 consumindo o caractere atual (tratando-o como o elemento central isolado de um palíndromo de comprimento ímpar).
- Avançar para q_1 por meio de uma transição vazia λ sem consumir o caractere atual (tratando o ponto corrente como o meio exato de um palíndromo par).

3.3 Decisões de Implementação

A memória auxiliar foi simulada usando a classe nativa `Stack<char>`. No caso não-determinístico (L3), o maior desafio de engenharia foi evitar que ramos computacionais inválidos corrompessem o estado da pilha dos ramos concorrentes legítimos.

Para solucionar essa restrição, foi projetado um algoritmo de busca com retrocesso (backtracking) recursivo através do método `ExplorarCaminhosL3`. Cada chamada recursiva gera um clone profundo e isolado da pilha utilizando a sub-rotina `CloneStack`. Se uma ramificação esvazia a pilha ao final do consumo da string, a recursão retorna verdadeiro, e o caminho bem-sucedido é impresso de forma reversa na tela.

3.4 Exemplos de Execução

Configurações Instantâneas no formato $\vdash [\text{estado}, \text{palavraRestante}, \text{pilha}]$:

Execução de L2 para a cadeia 'aabb':

```
[q0, aabb, Z]
⊢ [q0, abb, AZ]
⊢ [q0, bb, AAZ]
⊢ [q1, b, AZ]
⊢ [q1, λ, Z]
⊢ [q2, λ, λ] --> Pilha Vazia Detectada!
Resultado: ACEITA
```

Execução de L3 (Caminho de Sucesso) para a cadeia 'aba':

```
⊢ [q0, aba, Z]
⊢ [q1, ba, Z]      (Opção não-determinística: 'a' considerado o meio de
palíndromo ímpar)
⊢ [q1, a, AZ]
⊢ [q1, λ, Z]
⊢ [q2, λ, λ] --> Pilha Vazia Detectada via ramificação válida!
Resultado: ACEITA
```

3.5 Questões Reflexivas do Modelo

1. Qual a diferença fundamental entre o comportamento determinístico (L2) e o não-determinístico (L3) no código?

No comportamento determinístico de L2, cada par de entrada e topo da pilha possui exatamente uma transição possível; o fluxo é linear. No não-determinismo de L3, uma mesma configuração gera múltiplos caminhos válidos possíveis. No código, o determinismo é resolvido com um laço iterativo simples, enquanto o não-determinismo exige árvores de decisão

recursivas com custo computacional superior para explorar exaustivamente todas as alternativas de ramificação.

2. Como a pilha resolve o problema de contagem que o AFD não conseguia resolver?

A pilha funciona como um mecanismo de memória LIFO (Last-In, First-Out) potencialmente infinito. Ela substitui a necessidade de infinitos estados estáticos por armazenamento dinâmico. Ao empilhar símbolos durante a leitura do primeiro bloco de caracteres e desempilhá-los compassadamente no bloco subsequente, o autômato realiza operações aritméticas relacionais de subtração e igualdade por correspondência direta de um para um.

4. MÁQUINA DE TURING (MT)

4.1 Descrição da Linguagem (LMT)

A Máquina de Turing foi projetada para resolver um problema clássico de contagem tripla balanceada, classificado como uma Linguagem Sensível ao Contexto:

$$LMT = \{ a^n b^n c^n \mid n \geq 1 \}$$

Esta linguagem introduz uma restrição multidimensional impossível de ser validada por autômatos de pilha, visto que exige a verificação simultânea de igualdade entre três agrupamentos sequenciais distintos de caracteres.

4.2 Definição Formal

A máquina padrão implementada é descrita pela sêtupe $M_3 = (Q, \Sigma, \Gamma, \delta, q_0, _, F)$, onde:

- $Q: \{q_0, q_1, q_2, q_3, q_4, q_f\}$ (onde q_f é o estado de aceitação imediata).
- $\Sigma: \{a, b, c\}$

- $\Gamma: \{a, b, c, X, Y, Z, _ \}$ (onde X, Y, Z são marcadores de substituição e '_' é o Blank).
- δ (Função de Transição): $Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, L\}$

O algoritmo funciona convertendo iterativamente o primeiro 'a' encontrado em 'X', avançando para a direita no estado q1 até achar o primeiro 'b' (convertendo-o em 'Y'), avançando em q2 até achar o primeiro 'c' (convertendo-o em 'Z'). Em seguida, entra no estado de retorno à esquerda q3 até encontrar o último 'X' gravado, reiniciando o ciclo em q0. O estado q4 realiza a varredura de encerramento, garantindo que nenhum caractere órfão tenha restado na fita.

4.3 Decisões de Implementação

A fita infinita bidirecional foi modelada por meio de uma tabela hash dinâmica utilizando a classe Dictionary<int, char>. O índice inteiro atua diretamente como o cabeçote físico de leitura e escrita (cabeça). Ao contrário de estruturas estáticas como arrays ou strings tradicionais, o dicionário permite o crescimento elástico da fita para ambas as direções (índices negativos e positivos) sem estourar limites de memória física pré-alocada.

Para contornar o Problema da Parada (Halting Problem) intrínseco às Máquinas de Turing universais, foi adicionado um contador de segurança com teto fixado em `limitePassos = 1000`. Caso uma cadeia induza a máquina a um laço infinito, o sistema aborta a operação de forma segura, reportando rejeição por estouro de processamento.

4.4 Exemplos de Execução

Abaixo estão representados os passos de computação da fita para a cadeia ideal 'abc'. O cabeçote de leitura é indicado visualmente pelos colchetes []:

```

q0 | Fita: _[a]bc_
├ q1 | Fita: _X[b]c_
├ q2 | Fita: _XY[c]_
├ q3 | Fita: _X[Y]Z_
├ q3 | Fita: _[X]YZ_
├ q0 | Fita: _X[Y]Z_
├ q4 | Fita: _XY[Z]_
├ q4 | Fita: _XYZ[_]

```

⊢ qf | Fita: _XYZ_[_]

Resultado: ACEITA (Cadeia Processada em 8 transições legítimas)

4.5 Questões Reflexivas do Modelo

1. Por que esta linguagem ($a^n b^n c^n$) não pode ser reconhecida por um Autômato de Pilha?

Um autômato de pilha convencional possui apenas um canal de acesso à sua memória auxiliar (o topo). Ao processar e comparar os 'b's com os 'a's, o AP é obrigado a esvaziar os elementos da pilha para realizar a contagem de decréscimo. Uma vez consumidos os caracteres estruturais, a informação sobre o valor original de n é irremediavelmente destruída, impedindo que a máquina possua uma referência de memória preservada para validar o terceiro bloco constituído pelos caracteres 'c'.

2. De que forma a Máquina de Turing se assemelha e difere de um computador moderno baseado na Arquitetura de Von Neumann?

A semelhança reside no princípio da Computabilidade Universal. Ambos possuem uma unidade de controle de estados lógicos e uma memória de leitura/escrita irrestrita, o que os torna equivalentes na capacidade de executar qualquer algoritmo computável existente. A diferença reside no mecanismo de acesso à memória: a Máquina de Turing utiliza acesso estritamente sequencial e indexado pelo deslocamento físico do cabeçote (passo a passo para a esquerda ou direita), enquanto a Arquitetura de Von Neumann implementa Memória de Acesso Aleatório (RAM), permitindo a leitura e escrita direta em qualquer endereço físico instantaneamente através de barramentos de dados.

5. CONCLUSÃO

O desenvolvimento prático deste projeto propiciou a consolidação empírica dos conceitos abstratos discutidos na disciplina de Fundamentos Teóricos da Computação. Através da codificação em C#, tornou-se evidente o impacto direto que a arquitetura de dados e o gerenciamento de memória exercem sobre a capacidade de resolução de problemas de software.

Ao estruturar o AFD, observou-se a simplicidade e a eficiência operacional de sistemas puramente reativos, ideais para análise léxica e validação de padrões rígidos. O Autômato de Pilha evidenciou os desafios do não-determinismo, demonstrando como a recursão e o isolamento de estados são ferramentas fundamentais para simular árvores de busca complexas em problemas gramaticais estruturados. Por fim, a Máquina de Turing destacou-se como o modelo definitivo de computação clássica, explicitando como a capacidade de modificar livremente os dados de entrada em uma fita infinita expande os horizontes da programação para além das barreiras das linguagens livres de contexto.

O projeto cumpre com sucesso todos os requisitos técnicos estipulados, demonstrando que a teoria dos autômatos permanece como a base conceitual indispensável para o design de compiladores, interpretadores e sistemas de automação de software modernos.

6. REFERÊNCIAS BIBLIOGRÁFICAS

SIPSER, Michael. Introduction to the Theory of Computation. 3. ed. Boston: Cengage Learning, 2013.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.

MENEZES, Paulo Blauth. Linguagens Formais e Autômatos. 6. ed. Porto Alegre: Bookman, 2010.